

CTUC103 - Introduction to Object Oriented Programming

B.Sc. IT Semester - II

Practical Assignment – 6

Exception Handling, Managing IO and File Handling

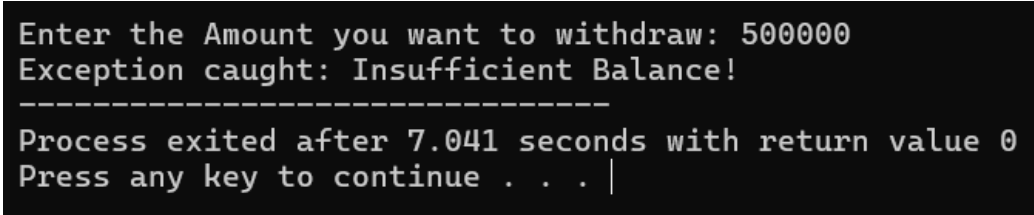
24BSIT029

P-1	Write a C++ program that takes two numbers as input from the user and performs division. If the denominator is zero, throw an exception and handle it appropriately. Hint: Use try, catch, and throw to handle the zero-division error.
Code:	<pre>#include<iostream> using namespace std; int main() { int numerator; int denominator; int result; cout<<"Enter Numerator: "; cin>>numerator; cout<<"Enter Denominator: "; cin>>denominator; try { if (denominator==0) { throw "Division by zero error!"; } result=numerator/denominator; cout<<"Result: "<<result; } catch (const char* errorMessage) { cout<<"Exception caught: " <<errorMessage; } return 0; }</pre>

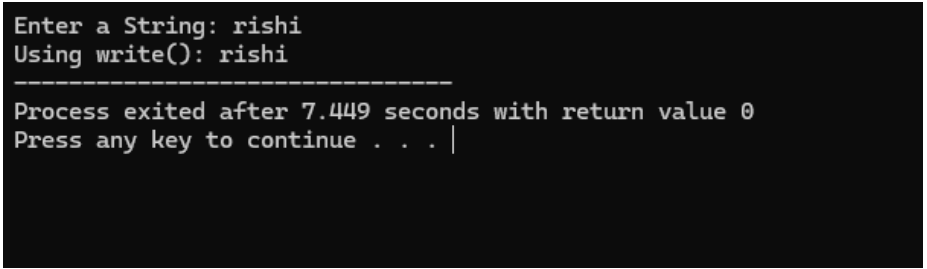
Output:	<pre> Enter Numerator: 2 Enter Denominator: 0 Exception caught: Division by zero error! ----- Process exited after 5.101 seconds with return value 0 Press any key to continue . . . </pre>
P-2	Write a C++ program that takes an array of five integers and asks the user to enter an index to access an element. If the index is out of bounds, throw an exception and handle it. Hint: Check if the index is within the valid range before accessing the array.
Code:	<pre> #include<iostream> using namespace std; int main() { int arr[5]; int i; int index; cout<<"Enter 5 integers:\n"; for(i=0;i<5;i++) { cin>>arr[i]; } cout<<"\nEnter an index to access an element: "; cin>>index; try { if(index<0 index>=5) { throw "Index out of bounds!"; } cout<<"Element at index "<<index<<" is: "<<arr[index]; } catch (const char* errorMessage) { cout<<"Exception caught: "<<errorMessage; } return 0; } </pre>

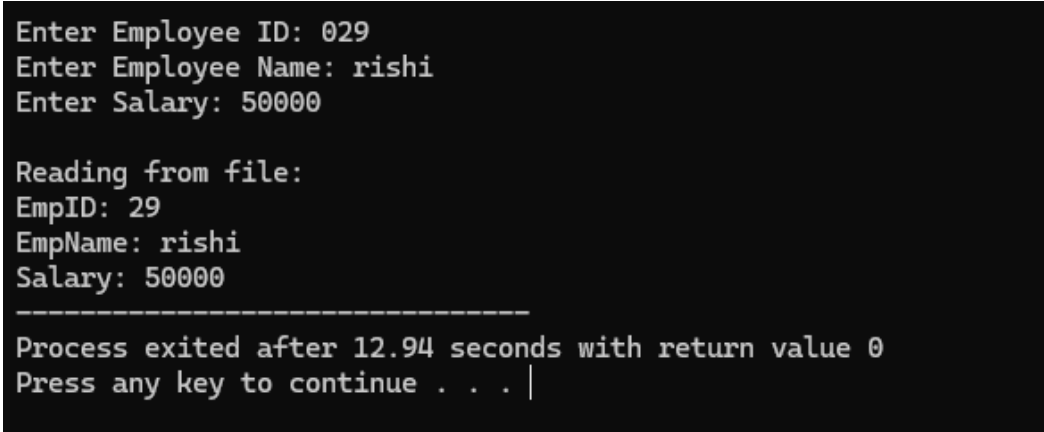
Output:	<pre> Enter 5 integers: 1 2 3 4 5 Enter an index to access an element: 5 Exception caught: Index out of bounds! ----- Process exited after 9.902 seconds with return value 0 Press any key to continue . . . </pre>
P-3	Write a program that asks the user to enter their age. If the age is negative or greater than 150, throw an exception and handle it. Otherwise, print the valid age. Hint: Use exception handling to ensure only valid age values are accepted.
Code:	<pre> #include<iostream> using namespace std; int main() { int Age; cout<<"Enter Your Age: "; cin>>Age; try { if(Age<0 Age>=150) { throw "Invalid Age entered!"; } else { cout<<"Your Age: "<<Age; } } catch (const char* error) { cout<<"Exception caught: "<<error; } return 0; } </pre>

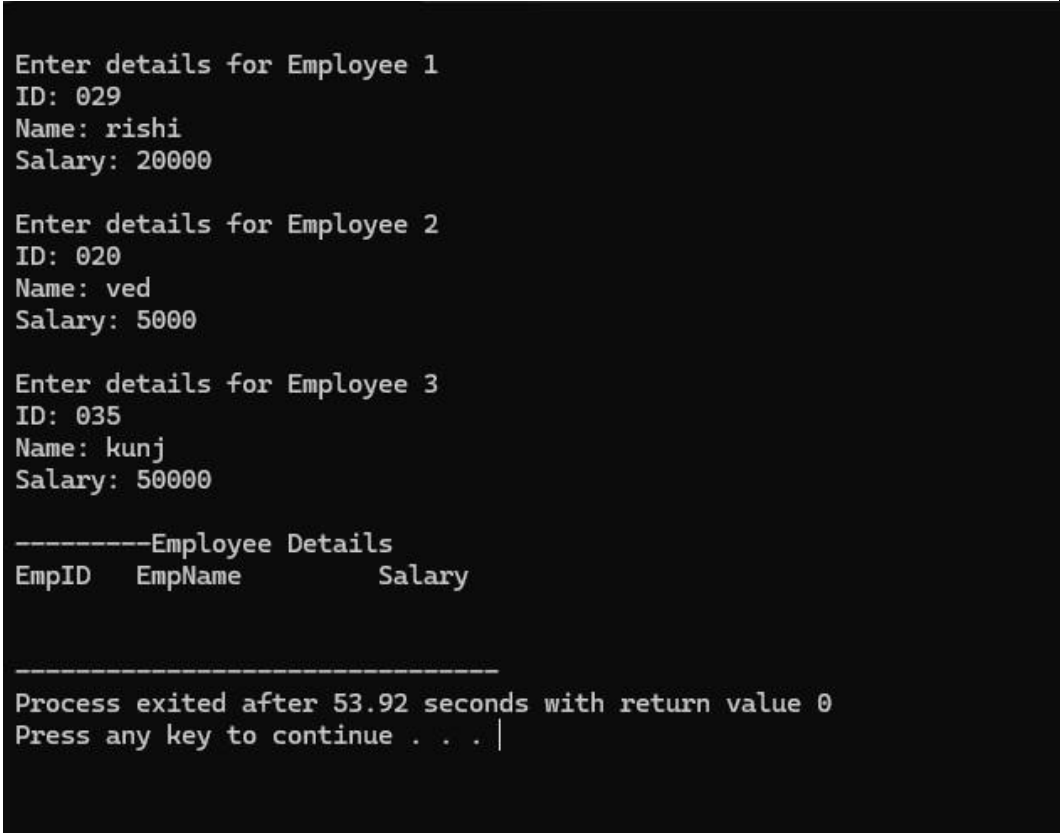
Output:	<pre> Enter Your Age: -19 Exception caught: Invalid Age entered! ----- Process exited after 8.815 seconds with return value 0 Press any key to continue . . . </pre>
P-4	Create a BankAccount class with a function withdraw(amount). If the withdrawal amount is greater than the balance, throw an exception. Handle the exception by displaying an appropriate message. Hint: Use class methods with exception handling.
Code:	<pre> #include<iostream> using namespace std; class BankAccount { private: int Balance; public: BankAccount(int b) { Balance=b; } void withdraw(int amount) { try { if(amount>Balance) { throw "Insufficient Balance!"; } else { cout<<"Withdrawal Successful"; } } catch(const char* error) { cout<<"Exception caught: "<<error; } } }; int main() { int amt; cout<<"Enter the Amount you want to withdraw: "; cin>>amt; BankAccount ba(5000); ba.withdraw(amt); } </pre>

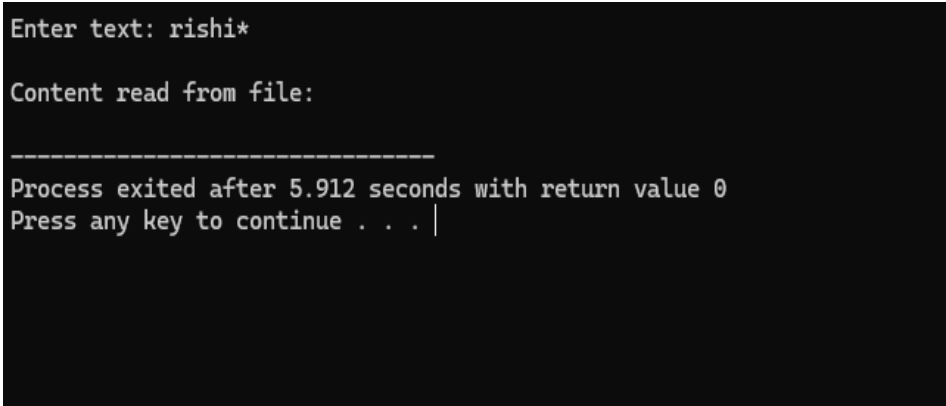
	<pre> return 0; } </pre>
Output:	 <pre> Enter the Amount you want to withdraw: 500000 Exception caught: Insufficient Balance! ----- Process exited after 7.041 seconds with return value 0 Press any key to continue . . . </pre>
P-5	Write a C++ program where a user enters marks (out of 100) for a student. If the marks are negative or greater than 100, throw a custom exception. Hint: Create a custom exception class to handle invalid marks.
Code:	<pre> #include<iostream> using namespace std; class Marks { public: const char* getMessage() //const { return "Invalid marks entered! Marks should be between 0 and 100."; } }; int main() { int marks; cout<<"Enter Marks: "; cin>>marks; try { if(marks<0 marks>100) { throw Marks(); } else { cout<<"Your Marks: "<<marks; } } catch(Marks m) { cout<<"Exception caught: "<<m.getMessage(); } return 0; } </pre>

	}
Output:	<pre> Enter Marks: 150 Exception caught: Invalid marks entered! Marks should be between 0 and 100. ----- Process exited after 6.379 seconds with return value 0 Press any key to continue . . . </pre>
P-6	Write a program to demonstrate the use of cout.width(), cout.precision(), and cout.fill().
Code:	<pre> #include<iostream> using namespace std; int main() { int num=123; cout.fill('*'); cout.width(10); cout<<num<<endl; cout.precision(2); cout<<"Precision 2: "<<num<<endl; cout.precision(5); cout<<"Precision 5: "<<num<<endl; cout.fill('-'); cout.width(12); cout<<num<<endl; return 0; } </pre>
Output:	<pre> *****123 Precision 2: 123 Precision 5: 123 -----123 ----- Process exited after 0.1905 seconds with return value 0 Press any key to continue . . . </pre>
P-7	Write a program to demonstrate the use of getline() and write() methods for input and output operations.
Code:	<pre> #include<iostream> #include<string.h> using namespace std; int main() </pre>

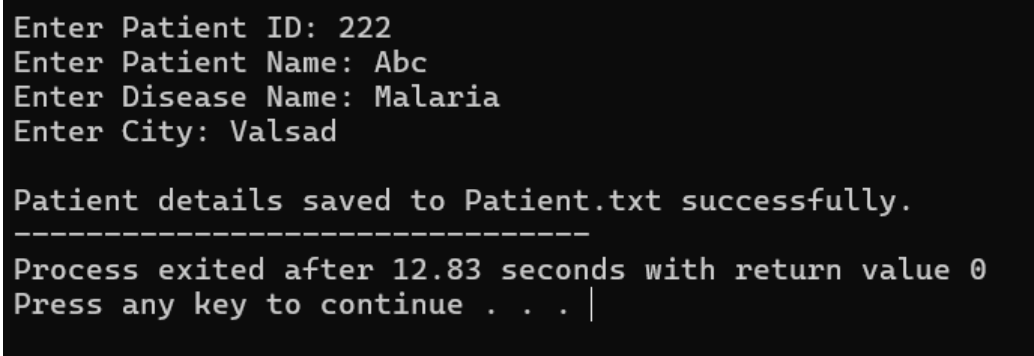
	<pre> { char str[50]; cout<<"Enter a String: "; cin>>str; cout<<"Using write(): "; cout.write(str,strlen(str)); return 0; } </pre>
Output:	 <pre> Enter a String: rishi Using write(): rishi ----- Process exited after 7.449 seconds with return value 0 Press any key to continue . . . </pre>
P-8	Write a program to store Employee details (EmpID, EmpName, Salary) into a file and then read the content from the file.
Code:	<pre> #include<iostream> #include<fstream> using namespace std; int main() { int EmpID; char EmpName[50]; float Salary; ofstream fout("C:\\Users\\rishi\\OneDrive\\Desktop\\employee.txt"); cout<<"Enter Employee ID: "; cin>>EmpID; cout<<"Enter Employee Name: "; cin>>EmpName; cout<<"Enter Salary: "; cin>>Salary; fout<<EmpID<<" "<<EmpName<<" "<<Salary; fout.close(); ifstream fin("C:\\Users\\rishi\\OneDrive\\Desktop\\employee.txt"); cout<<"\nReading from file:\n"; fin>>EmpID>>EmpName>>Salary; cout<<"EmpID: "<<EmpID<<"\nEmpName: "<<EmpName<<"\nSalary: "<<Salary; </pre>

	<pre> fin.close(); return 0; } </pre>
Output:	 <pre> Enter Employee ID: 029 Enter Employee Name: rishi Enter Salary: 50000 Reading from file: EmpID: 29 EmpName: rishi Salary: 50000 ----- Process exited after 12.94 seconds with return value 0 Press any key to continue . . . </pre>
P-9	Write a program to store details of three Employee (EmpID, EmpName, Salary) into a file and displays the content in a properly formatted manner.
Code:	<pre> #include<iostream> #include<fstream> using namespace std; int main() { int EmpID; char EmpName[50]; float Salary; ofstream fout("C:\\Users\\rishi\\OneDrive\\Desktop\\employees_Details.txt"); for(int i=0;i<3;i++) { cout<<"\nEnter details for Employee "<<i+1<<"\n"; cout<<"ID: "; cin>>EmpID; cout<<"Name: "; cin>>EmpName; cout<<"Salary: "; cin>>Salary; fout<<EmpID<<" "<<EmpName<<" "<<Salary<<"\n"; } fout.close(); ifstream fin("C:\\Users\\rishi\\OneDrive\\Desktop\\employees_Details.txt"); cout<<"\n-----Employee Details ----- \n"; cout<<"EmpID\tEmpName\t\tSalary\n"; cout<<"-----\n"; </pre>

	<pre> while(fin>>EmpID>>EmpName>>Salary) { cout<<EmpID<<"\t"<<EmpName<<"\t\t"<<Salary<<"\n"; } fin.close(); return 0; } </pre>
Output:	 <pre> Enter details for Employee 1 ID: 029 Name: rishi Salary: 20000 Enter details for Employee 2 ID: 020 Name: ved Salary: 5000 Enter details for Employee 3 ID: 035 Name: kunj Salary: 50000 -----Employee Details EmpID EmpName Salary ----- Process exited after 53.92 seconds with return value 0 Press any key to continue . . . </pre>
P-10	Implement a program to read from and write to a file character by character using get() and put() methods.
Code:	<pre> #include<iostream> #include<fstream> using namespace std; int main() { char ch; ofstream fout("C:\\\\Users\\rishi\\OneDrive\\Desktop\\char_file.txt"); cout<<"Enter text: "; while(cin.get(ch)&&ch!='*') { fout.put(ch); } fout.close(); } </pre>

	<pre> ifstream fin("C:\\Users\\rishi\\OneDrive\\Desktop\\char_file.txt"); cout<<"\nContent read from file:\n"; while(fin.get(ch)) { cout<<ch; } fin.close(); return 0; } </pre>
Output:	 <pre> Enter text: rishi* Content read from file: ----- Process exited after 5.912 seconds with return value 0 Press any key to continue . . . </pre>
P-11	Write a program to create a file named "Product.txt" and store the following details: • Product_Name, Product_Price, Batch_Number.
Code:	<pre> #include<iostream> #include<fstream> using namespace std; int main() { char Product_Name[50]; float Product_Price; int Batch_Number; ofstream fout("C:\\Users\\Charmi\\OneDrive\\Desktop\\Product.txt"); cout<<"Enter Product Name: "; cin>>Product_Name; cout<<"Enter Product Price: "; cin>>Product_Price; cout<<"Enter Batch Number: "; cin>>Batch_Number; fout<<Product_Name<<" "<<Product_Price<<" "<<Batch_Number<<"\n"; fout.close(); cout<<"\nData written to Product.txt successfully."; return 0; } </pre>

Output:	<pre> Enter Product Name: Chocolate Enter Product Price: 50 Enter Batch Number: 10 Data written to Product.txt successfully. ----- Process exited after 34.9 seconds with return value 0 Press any key to continue . . . </pre>
P-12	Write a program to append the following details to the existing file "Product.txt". • Product_Type, Product_Weight
Code:	<pre> #include<iostream> #include<fstream> using namespace std; int main() { char Product_Type[50]; float Product_Weight; ofstream fout("C:\\Users\\Charmi\\OneDrive\\Desktop\\Product.txt",ios::app); cout<<"Enter Product Type: "; cin>>Product_Type; cout<<"Enter Product Weight: "; cin>>Product_Weight; fout<<Product_Type<<" "<<Product_Weight<<"\n"; fout.close(); cout<<"\nDetails appended to Product.txt successfully."; return 0; } </pre>
Output:	<pre> Enter Product Type: Sweet Enter Product Weight: 59.99 Details appended to Product.txt successfully. ----- Process exited after 82.02 seconds with return value 0 Press any key to continue . . . </pre>
P-13	Write a program to create a file named "Patient.txt" and record the following details: • Patient ID, Patient Name, Disease Name, City.
Code:	<pre> #include<iostream> </pre>

	<pre> #include<fstream> using namespace std; int main() { int Patient_ID; char Patient_Name[50], Disease_Name[50], City[50]; ofstream fout("C:\\Users\\Charmi\\OneDrive\\Desktop\\Patient.txt"); cout<<"Enter Patient ID: "; cin>>Patient_ID; cout<<"Enter Patient Name: "; cin>>Patient_Name; cout<<"Enter Disease Name: "; cin>>Disease_Name; cout<<"Enter City: "; cin>>City; fout<<Patient_ID<<" "<<Patient_Name<<" "<<Disease_Name<<" "<<City<<"\\n"; fout.close(); cout<<"\\nPatient details saved to Patient.txt successfully."; return 0; } </pre>
Output:	 <pre> Enter Patient ID: 222 Enter Patient Name: Abc Enter Disease Name: Malaria Enter City: Valsad Patient details saved to Patient.txt successfully. ----- Process exited after 12.83 seconds with return value 0 Press any key to continue . . . </pre>
P-14	Write a program to append the following details to the to the “Patient.txt” file: • Doctor_Name, Doctor_Speciality After appending, read and display all the patient details on the console.
Code:	<pre> #include<iostream> #include<fstream> using namespace std; int main() { char Doctor_Name[50], Doctor_Speciality[50]; ofstream fout("C:\\Users\\Charmi\\OneDrive\\Desktop\\Patient.txt",ios::app); </pre>

	<pre> cout<<"Enter Doctor Name: "; cin>>Doctor_Name; cout<<"Enter Doctor Speciality: "; cin>>Doctor_Speciality; fout<<Doctor_Name<<" "<<Doctor_Speciality<<"\n"; fout.close(); cout<<"\n-----Contents of Patient.txt ----- \n"; ifstream fin("C:\\Users\\Charmi\\OneDrive\\Desktop\\Patient.txt"); char ch; while(fin.get(ch)) { cout<<ch; } fin.close(); return 0; } </pre>
Output:	<pre> Enter Doctor Name: Dr.Z Enter Doctor Speciality: Infection_sp -----Contents of Patient.txt----- 222 Abc Malaria Valsad Dr.Z Infection_sp ----- Process exited after 11.24 seconds with return value 0 Press any key to continue . . . </pre>